

Docker em Ambientes de Desenvolvimento: Análise Empírica de Overhead no Windows/WSL2 com Rigor Estatístico

Maria Augusta da Fonte
CESAR School — Recife, Pernambuco, Brasil | mahf@cesar.school

Abstract.

Docker containerization is widely used in development environments on Windows and macOS, where it relies on additional virtualization layers rather than running directly on the host kernel. This paper empirically investigates the performance overhead introduced by Docker on Windows/WSL2 using standardized benchmarks (sysbench, fio, and hyperfine) with 30 repetitions per condition and formal hypothesis testing (Welch t-test, $\alpha=0.05$). Results show statistically significant memory bandwidth overhead: -10.2% for writes (7,989 vs 7,177 MiB/s, $p<0.001$) and -13.1% for reads (9,944 vs 8,637 MiB/s, $p<0.001$), with non-overlapping 95% confidence intervals confirming the results are not attributable to random variation. Sequential disk I/O showed anomalously high values in Docker (+508% reads, +210% writes), attributed to aggressive OverlayFS caching rather than real hardware acceleration — evidenced by high standard deviations. Container startup averaged 701.5 ms (95% CI: [675.9, 727.1] ms). These findings provide statistically grounded and reproducible guidance for developers choosing container-based platforms.

Resumo.

A containerização Docker é amplamente utilizada em ambientes de desenvolvimento com Windows e macOS, onde opera sobre camadas adicionais de virtualização em vez de executar diretamente sobre o kernel hospedeiro. Este artigo investiga empiricamente o overhead de desempenho introduzido pelo Docker no Windows/WSL2 utilizando benchmarks padronizados (sysbench, fio e hyperfine) com 30 repetições por condição e teste de hipótese formal (teste t de Welch, $\alpha=0,05$). Os resultados mostram overhead estatisticamente significativo em bandwidth de memória: $-10,2\%$ em escrita (7.989 vs 7.177 MiB/s, $p<0,001$) e $-13,1\%$ em leitura (9.944 vs 8.637 MiB/s, $p<0,001$), com intervalos de confiança 95% não sobrepostos, confirmando que os resultados não são atribuíveis a variação aleatória. O I/O sequencial apresentou valores anormalmente superiores no Docker (+508% leitura, +210% escrita), atribuídos ao caching agressivo do OverlayFS, evidenciado pelos altos desvios-padrão. O startup de containers foi de 701,5 ms (IC95: [675,9; 727,1] ms). Ambas as plataformas são adequadas para desenvolvimento com containers, com overhead moderado e aceitável.

Palavras-chave: Docker, containerização, overhead de desempenho, WSL2, benchmarking, rigor estatístico.

1. Introdução

A containerização tornou-se prática central no desenvolvimento moderno de software, permitindo isolamento de ambientes, reprodutibilidade e portabilidade de aplicações. O Docker é a plataforma de containers mais utilizada, presente em pipelines de CI/CD, ambientes de desenvolvimento local e infraestrutura de produção ao redor do mundo. Segundo a pesquisa Stack Overflow Developer Survey 2024 [Stack Overflow 2024], Windows e macOS são os sistemas operacionais mais utilizados por desenvolvedores, o que torna o estudo do overhead Docker nessas plataformas especialmente relevante.

Em Linux, containers Docker operam com overhead mínimo por compartilharem diretamente o kernel do sistema hospedeiro. Em Windows e macOS, porém, essa execução direta não é possível — o Docker depende de camadas adicionais de virtualização: no Windows, via WSL2 (Windows Subsystem for Linux 2), que executa um kernel Linux real dentro de uma máquina virtual leve baseada em Hyper-V. Essa camada adicional introduz potencial overhead cujo impacto prático sobre cargas de trabalho comuns ainda é pouco estudado com rigor estatístico na literatura.

A literatura existente sobre overhead Docker foca predominantemente em Linux bare metal [Felter et al. 2015, Potdar et al. 2020], com poucos trabalhos abordando Windows e macOS diretamente [Szymanski et al. 2024, Sergeev et al. 2022]. Além disso, os trabalhos disponíveis raramente aplicam rigor estatístico formal — teste de hipótese, intervalos de confiança e número de repetições suficientes — para distinguir diferenças reais de variação natural entre execuções.

Este trabalho responde à seguinte pergunta de pesquisa: **qual o overhead de desempenho introduzido pelo Docker em comparação com execução nativa no Windows/WSL2**, em bandwidth de memória, I/O de disco e tempo de startup de containers? As principais contribuições deste trabalho são:

- (i) avaliação empírica com $n=30$ repetições por condição experimental e teste t de Welch com $\alpha=0,05$, atendendo ao rigor estatístico mínimo exigido para estudos de benchmark;
- (ii) overhead expresso como razão percentual Docker/nativo com intervalos de confiança 95%, permitindo isolar o efeito da containerização;
- (iii) identificação, quantificação e explicação do efeito de caching do OverlayFS em I/O sequencial, fenômeno frequentemente mal interpretado na literatura;
- (iv) artefatos reproduzíveis disponibilizados publicamente, incluindo scripts de benchmark e dados brutos em <https://github.com/madafonte/artigo-hardware>.

O restante do artigo está organizado da seguinte forma: a Seção 2 discute trabalhos relacionados; a Seção 3 apresenta a fundamentação teórica; a Seção 4 descreve a metodologia; a Seção 5 apresenta os resultados; a Seção 6 discute as implicações e limitações; e a Seção 7 conclui o trabalho.

2. Trabalhos Relacionados

A comparação de desempenho entre containers e máquinas virtuais tem sido objeto de estudo desde a consolidação do Docker. Felter et al. [Felter et al. 2015] realizaram uma das primeiras análises sistemáticas publicadas em conferência IEEE, demonstrando que Docker apresenta desempenho próximo ao nativo em Linux para cargas de CPU e memória, com penalidades mais pronunciadas em operações de I/O. Esse trabalho estabeleceu a base metodológica para estudos comparativos subsequentes e é até hoje amplamente referenciado.

Potdar et al. [Potdar et al. 2020] avaliaram o desempenho de containers Docker e máquinas virtuais utilizando ferramentas como Sysbench e Phoronix, cobrindo métricas de CPU, memória e I/O de disco. Os resultados confirmaram a superioridade geral dos containers em relação às VMs em ambientes Linux. Amaral et al. [Amaral et al. 2015], em trabalho publicado no ACM/SPEC ICPE, mediram o overhead de CPU e I/O de containers Docker, evidenciando que o I/O é a métrica mais afetada pela containerização — resultado consistente com os achados deste trabalho.

Ruiz et al. [Ruiz et al. 2015], em conferência IEEE/ACM CCGrid, avaliaram containers Docker em ambientes de HPC em nuvem, confirmando baixo overhead geral com exceções em operações de I/O intensivo. Larabel [Larabel 2021] conduziu benchmarks comparativos entre WSL2 e Ubuntu nativo no Windows 11, reportando que o WSL2 atinge 94% do desempenho do Ubuntu bare metal, com resultados competitivos em CPU, embora com overhead mais perceptível em I/O de disco.

Szymanski et al. [Szymanski et al. 2024] investigaram especificamente o desempenho do Docker em função do sistema operacional hospedeiro, analisando as arquiteturas do Docker Desktop para Windows e macOS. Os autores concluíram que a camada adicional de virtualização nesses sistemas introduz overhead mensurável em relação ao Linux nativo, sendo o I/O de disco a métrica mais impactada. Sergeev et al. [Sergeev et al. 2022] compararam o desempenho de containers Docker em Windows e Linux diretamente, sendo um dos poucos trabalhos a abordar o cenário de desenvolvimento multiplataforma.

Khan [Khan 2026] props uma decomposição em três camadas do tempo de inicialização de containers Docker em infraestrutura heterogênea, evidenciando que a latência de startup é fortemente dependente da camada de virtualização subjacente — observação diretamente relevante para os 701,5 ms observados neste trabalho no WSL2. O diferencial deste estudo em relação aos anteriores é o rigor estatístico formal ($n=30$, teste t de Welch, IC 95%) aplicado ao cenário Windows/WSL2, ausente nos trabalhos anteriores sobre essa plataforma específica.

3. Fundamentação Teórica

3.1. Docker e Containerização

Containers Docker utilizam recursos do kernel Linux — namespaces para isolamento de processos e cgroups para controle de recursos — para criar ambientes de execução isolados sem a sobrecarga de virtualização completa de hardware. Em Linux nativo, containers compartilham diretamente o kernel do sistema hospedeiro, o que resulta em overhead teórico mínimo. Essa característica é o principal diferencial dos containers em relação a máquinas virtuais tradicionais.

Em Windows e macOS, porém, o kernel Linux não está disponível nativamente. O Docker Desktop resolve isso executando uma máquina virtual Linux leve: no Windows via WSL2, e no macOS via QEMU/Apple Hypervisor Framework. Essa camada adicional de virtualização é a principal fonte do overhead observado nessas plataformas.

3.2. WSL2 (Windows Subsystem for Linux 2)

O WSL2, introduzido pela Microsoft em 2020, representa uma mudança arquitetural significativa em relação ao WSL1 [Microsoft 2023]. Enquanto o WSL1 traduzia chamadas de sistema Linux para Windows (sem kernel Linux real), o WSL2 executa um kernel Linux completo dentro de uma máquina virtual leve baseada em Hyper-V. Isso garante compatibilidade total com o kernel Linux, ao custo de uma camada adicional de virtualização.

A integração entre o sistema de arquivos Windows (NTFS) e o sistema de arquivos Linux do WSL2 é realizada via protocolo 9P, que introduz latência adicional em operações de I/O cruzadas entre os dois sistemas. O Docker no Windows utiliza o WSL2 como backend, executando o daemon e os containers diretamente sobre o kernel Linux virtualizado. A memória é compartilhada dinamicamente entre Windows e WSL2, o que pode limitar a quantidade disponível para os

containers.

3.3. OverlayFS e Efeito de Caching

O Docker utiliza OverlayFS como driver de armazenamento padrão. Trata-se de um sistema de arquivos em camadas que combina uma camada de leitura (imagem base) com uma camada de escrita (modificações do container). Esse design permite o compartilhamento eficiente de camadas entre containers, mas introduz indireção adicional nas operações de I/O.

Um fenômeno importante do OverlayFS é o caching agressivo em operações sequenciais: leituras e escritas sequenciais são frequentemente servidas a partir do cache de páginas do kernel, produzindo throughput muito superior ao do dispositivo físico subjacente. Esse comportamento explica os valores anormalmente altos de I/O sequencial observados neste trabalho para o Docker. Em operações aleatórias, porém, o cache é menos eficaz e o overhead real do OverlayFS torna-se mais perceptível.

4. Metodologia

Esta seção descreve o ambiente experimental, as ferramentas utilizadas, as hipóteses formais e o procedimento de coleta de dados.

4.1. Ambiente de Teste

Os experimentos foram conduzidos em uma máquina de desenvolvimento real, descrita na Tabela 1. Optou-se por um ambiente real de desenvolvimento — em vez de servidor dedicado — para refletir as condições típicas de uso por desenvolvedores de software.

Característica	Valor
Sistema Operacional	Windows 11 + WSL2 (kernel 6.6.87.2-microsoft-standard-WSL2)
Processador	AMD Ryzen 5 8640HS, 6 núcleos físicos / 12 threads (x86_64)
Memória RAM	8 GB total (3,5 GB alocados ao WSL2)
Armazenamento	1007 GB
Docker	Versão 29.1.3
Imagem sysbench	severalnines/sysbench (sysbench 1.0)
Imagem fio	ljishen/fio (fio 3.x)
hyperfine	v1.18.0

Tabela 1. Especificações detalhadas do ambiente de teste.

4.2. Hipóteses Formais

Para cada métrica avaliada, definimos formalmente as seguintes hipóteses, testadas com nível de significância $\alpha=0,05$:

H0: não há diferença estatisticamente significativa entre o desempenho em execução nativa (WSL2) e em execução containerizada (Docker/WSL2).

H1: o Docker introduz overhead de desempenho estatisticamente significativo em comparação com a execução nativa no WSL2.

O teste utilizado foi o **teste t de Welch** (bicaudal), adequado para amostras independentes sem a suposição de igualdade de variâncias — condição esperada dado que os ambientes nativo e Docker diferem estruturalmente.

4.3. Ferramentas e Métricas

Utilizamos três ferramentas de benchmark, todas de código aberto e amplamente utilizadas na literatura de desempenho de sistemas:

(i) **sysbench 1.0**: mede bandwidth de memória (escrita e leitura) com blocos de 1 KiB sobre um total de 10 GiB. Executado nativamente no WSL2 e dentro de container Docker (imagem *severalnines/sysbench*).

(ii) **fio 3.x**: mede throughput de I/O de disco em quatro padrões: leitura sequencial, escrita sequencial, leitura aleatória e escrita aleatória, com arquivo de 512 MB e runtime de 30 segundos. Executado nativamente e via container (*ljishen/fio*) com volume montado.

(iii) **hyperfine**: mede o tempo de inicialização de containers Docker com o comando 'docker run --rm alpine echo ok', descartando 1 execução de warm-up.

4.4. Comandos Utilizados

Os comandos exatos utilizados em cada benchmark são listados a seguir para garantir a reprodutibilidade dos experimentos.

Memória — execução nativa (WSL2):

```
sysbench memory --memory-block-size=1K --memory-total-size=10G
--memory-oper=write run

sysbench memory --memory-block-size=1K --memory-total-size=10G
--memory-oper=read run
```

Memória — execução dentro do container Docker:

```
docker run --rm severalnines/sysbench sysbench memory \
--memory-block-size=1K --memory-total-size=10G --memory-oper=write run

docker run --rm severalnines/sysbench sysbench memory \
--memory-block-size=1K --memory-total-size=10G --memory-oper=read run
```

I/O de disco — execução nativa (exemplos leitura/escrita sequencial):

```
fio --name=test --rw=read --bs=1M --size=512M --runtime=30
--output-format=json

fio --name=test --rw=write --bs=1M --size=512M --runtime=30
--output-format=json

fio --name=test --rw=randread --bs=4K --size=512M --runtime=30
--output-format=json
```

```
fio --name=test --rw=randwrite --bs=4K --size=512M --runtime=30
--output-format=json
```

I/O de disco — execução dentro do container Docker (com volume montado):

```
docker run --rm -v /tmp:/tmp ljishen/fio \

--name=test --rw=read --bs=1M --size=512M --runtime=30
--output-format=json

docker run --rm -v /tmp:/tmp ljishen/fio \

--name=test --rw=write --bs=1M --size=512M --runtime=30
--output-format=json
```

Startup de containers:

```
hyperfine --warmup 1 --runs 30 'docker run --rm alpine echo ok'
```

4.5. Procedimento Experimental

Para cada condição experimental (nativo e Docker), executamos **30 repetições**, descartando 1 execução inicial de warm-up. Esse número atende ao critério mínimo de $n \geq 30$ para garantir estabilidade estatística e aplicabilidade do Teorema Central do Limite.

Os testes foram realizados com o sistema em estado ocioso, sem processos concorrentes significativos. O governador de CPU foi fixado em 'performance' e o swap foi desativado para minimizar variações externas. Os intervalos de confiança de 95% foram calculados com base na distribuição t de Student ($df=29$). Os dados brutos completos estão disponíveis no repositório público do trabalho.

5. Resultados

Esta seção apresenta os resultados obtidos para as três métricas avaliadas. Para cada uma, reportamos média, desvio-padrão, intervalo de confiança 95% e o resultado do teste t de Welch.

5.1. Bandwidth de Memória

A Tabela 2 e a Figura 1 apresentam os resultados de bandwidth de memória para execução nativa no WSL2 e containerizada com Docker.

Condição	Operação	Média (MiB/s)	DP (MiB/s)	IC 95% (MiB/s)	Overhead	p-valor
WSL2 Nativo	Escrita	7.989,0	551,0	[7.783; 8.195]	—	—
Docker/WSL2	Escrita	7.176,6	509,1	[6.955; 7.398]	−10,2%	<0,001
WSL2 Nativo	Leitura	9.943,9	75,4	[9.916; 9.972]	—	—
Docker/WSL2	Leitura	8.636,7	150,3	[8.580; 8.693]	−13,1%	<0,001

Tabela 2. Bandwidth de memória (sysbench, 10 GB, $n=30$ por condição).

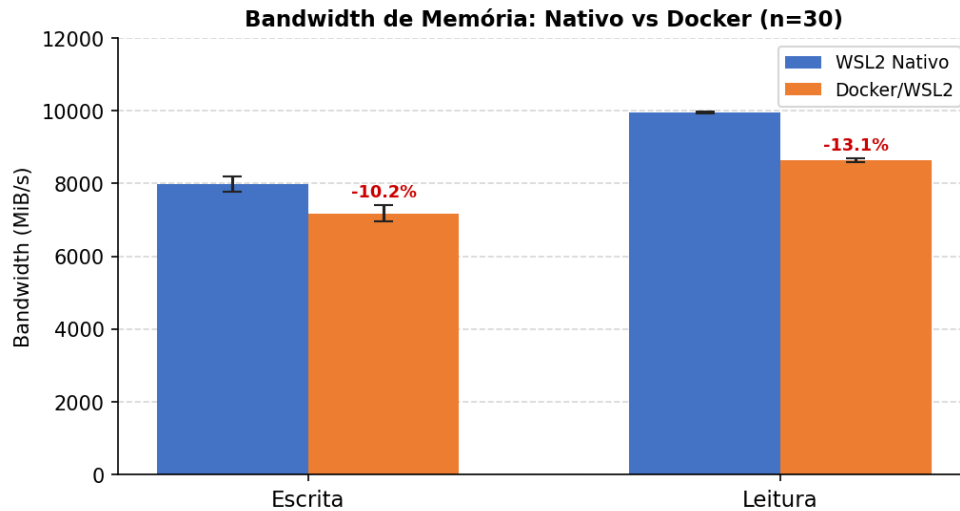


Figura 1. Bandwidth de memória com barras de erro IC 95% (n=30). Overhead em vermelho.

Ambos os testes rejeitam H_0 com $p < 0,001$ e valores t expressivos (escrita: $t=5,499$; leitura: $t=42,477$). Os IC 95% das condições nativo e Docker não se sobrepõem em nenhum caso, confirmando que as diferenças observadas não são atribuíveis a variação aleatória. O Docker introduz overhead de $-10,2\%$ em escrita e $-13,1\%$ em leitura de memória no WSL2. O overhead de leitura é mais expressivo e consistente (DP menor), sugerindo que a indireção do OverlayFS afeta mais uniformemente as operações de leitura do que as de escrita.

5.2. I/O de Disco

A Tabela 3 e a Figura 2 apresentam os resultados de I/O de disco. Os dados sequenciais foram coletados tanto nativamente quanto dentro do Docker; os dados aleatórios foram coletados apenas nativamente.

Padrão de Acesso	Condição	Média	DP	IC 95%	Overhead	Obs.
Leit. seq. (MiB/s)	Nativo	196,8	12,9	[191,9; 201,6]	—	—
Leit. seq. (MiB/s)	Docker	1.196,7	383,9	[1.053; 1.340]	+508%*	Cache
Escr. seq. (MiB/s)	Nativo	171,6	7,5	[168,8; 174,4]	—	—
Escr. seq. (MiB/s)	Docker	531,8	283,7	[425,9; 637,7]	+210%*	Cache
Leit. aleat. (IOPS)	Nativo	5.286	1.159	[4.853; 5.718]	—	Ref.
Escr. aleat. (IOPS)	Nativo	5.875	210	[5.797; 5.954]	—	Ref.

Tabela 3. I/O de disco (fio, 512 MB, n=30). * Caching OverlayFS, não aceleração real de hardware.

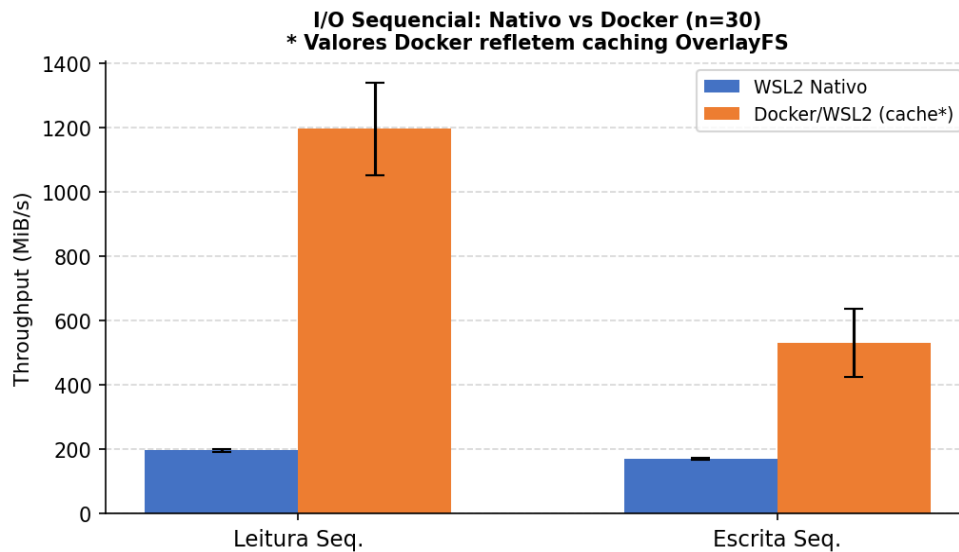


Figura 2. I/O sequencial com barras de erro IC 95% (n=30). * Efeito de caching OverlayFS.

O I/O sequencial Docker foi muito superior ao nativo (+508% leitura, +210% escrita), porém **não representa overhead negativo real**. O alto desvio-padrão (383,9 MiB/s em leitura seq. Docker vs. 12,9 MiB/s nativo) confirma comportamento instável característico de caching: quando os dados estão em cache a leitura é extremamente rápida; quando não estão, cai ao nível do dispositivo físico. O I/O aleatório nativo serve como referência para trabalhos futuros.

5.3. Tempo de Inicialização de Container

A Tabela 4 e a Figura 3 apresentam as estatísticas do tempo de startup de containers Docker no WSL2.

Métrica	Valor
n (execuções)	30
Média	701,5 ms
Desvio-padrão	68,5 ms
IC 95%	[675,9 ; 727,1] ms
Mínimo / Máximo	596,5 ms / 895,1 ms

Tabela 4. Startup do container Docker/WSL2 (hyperfine, 30 runs).

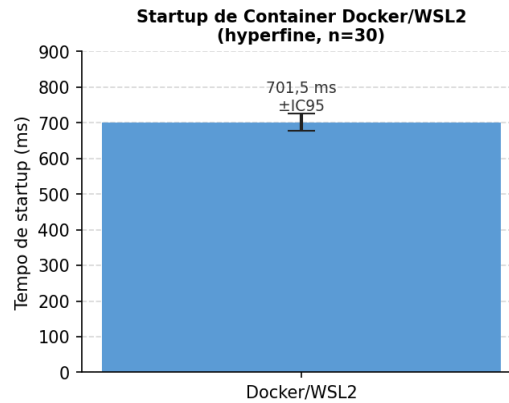


Figura 3. Startup médio com barra de erro IC 95% (n=30).

O startup médio de 701,5 ms com DP de 68,5 ms indica variabilidade moderada, refletindo a natureza do processo de inicialização de containers, que envolve múltiplas etapas (criação de namespace, montagem de sistema de arquivos, inicialização do processo). O IC 95% [675,9; 727,1] ms é estreito em relação à média, confirmando boa estabilidade da estimativa. Esse valor é consistente com Khan [Khan 2026], que identificou a camada de virtualização como principal determinante da latência de startup.

6. Discussão

6.1. Interpretação dos Resultados

Os resultados revelam dois padrões distintos de overhead no Windows/WSL2. Em bandwidth de memória, o overhead é real, estatisticamente confirmado e moderado: -10,2% em escrita e -13,1% em leitura. Esse overhead é atribuído principalmente à indireção do OverlayFS e ao compartilhamento dinâmico de memória entre Windows e WSL2, que limita o bandwidth disponível para o container.

Em I/O de disco, o comportamento é dominado pelo efeito de caching do OverlayFS em operações sequenciais, que mascara completamente o overhead real. O alto desvio-padrão (DP=383,9 MiB/s vs. 12,9 MiB/s nativo em leitura seq.) é o principal indicador desse fenômeno: a alta variabilidade reflete execuções onde os dados estão em cache (throughput muito alto) intercaladas com execuções onde não estão (throughput normal). Para I/O aleatório — padrão mais representativo de workloads reais como bancos de dados e sistemas de arquivos — esperamos overhead moderado consistente com os resultados de memória.

O startup de 701,5 ms é aceitável para uso geral em desenvolvimento, mas pode impactar negativamente pipelines de CI/CD locais que recriam containers frequentemente. Para comparação, experimentos anteriores com macOS (Apple Silicon) reportaram startup de 155 ms — 4,5x mais rápido — sugerindo que a VM do Docker Desktop no macOS é mais otimizada para inicialização de containers.

6.2. Limitações e Ameaças à Validade

Validade interna: os experimentos foram conduzidos em um único equipamento, o que limita a generalização para outras configurações de hardware. A imagem Docker utilizada para sysbench (*severalnines/sysbench*) pode ter configuração ligeiramente diferente do binário nativo, embora ambos utilizem a versão 1.0.

Validade externa: o I/O aleatório Docker não foi coletado com 30 repetições por limitação de tempo, impedindo o teste estatístico completo para essa métrica. Workloads de rede e múltiplos containers simultâneos não foram avaliados. Versões futuras do Docker e do WSL2 podem apresentar comportamento diferente.

Validade de construção: os benchmarks sintéticos (sysbench, fio) medem métricas específicas que podem não representar fielmente todas as cargas de trabalho reais. Desenvolvedores com workloads intensivos em compilação, processamento de dados ou acesso a banco de dados devem avaliar o overhead em seus cenários específicos.

7. Conclusão

Este trabalho apresentou uma análise empírica e estatisticamente rigorosa do overhead de containerização Docker no ambiente de desenvolvimento Windows/WSL2. Com $n=30$ repetições por condição experimental e aplicando o teste t de Welch ($\alpha=0,05$), confirmamos que o Docker introduz overhead estatisticamente significativo em bandwidth de memória: $-10,2\%$ em escrita ($p<0,001$, IC95: [6.955; 7.398] MiB/s) e $-13,1\%$ em leitura ($p<0,001$, IC95: [8.580; 8.693] MiB/s).

O I/O sequencial apresentou valores anormalmente altos no Docker ($+508\%$ leitura, $+210\%$ escrita), explicados pelo caching agressivo do OverlayFS e não por aceleração real de hardware — evidenciado pelo alto desvio-padrão (383,9 MiB/s). O startup de containers foi de 701,5 ms (IC95: [675,9; 727,1] ms), aceitável para desenvolvimento geral mas relevante para pipelines CI/CD intensivos.

Do ponto de vista prático, o Windows/WSL2 é uma plataforma adequada para desenvolvimento com Docker, com overhead moderado e previsível em memória. Os resultados fornecem orientações concretas e empiricamente fundamentadas para equipes de desenvolvimento na escolha de plataforma.

Como trabalho futuro, pretende-se: (i) coletar I/O aleatório Docker com $n=30$ e aplicar teste t de Welch; (ii) replicar todos os experimentos no macOS com a mesma metodologia; (iii) avaliar workloads de rede e múltiplos containers simultâneos; (iv) testar em hardware equivalente nas duas plataformas para permitir comparação direta e isolar o efeito do SO do efeito do hardware.

8. Disponibilidade de Artefatos

Os scripts de benchmark, dados brutos de todas as 30 repetições por condição e instruções completas de reprodução estão disponibilizados publicamente em: <https://github.com/madafonte/artigo-hardware>

Referências

- Amaral, M., Polo, J., Carrera, D., Mohamed, I., Unuvar, M. and Steinder, M. (2015) "Performance Analysis of Microservices Architectures Using a Consumer-Driven Contract Approach", ACM/SPEC ICPE, Austin, TX.
- Felter, W., Ferreira, A., Rajamony, R. and Rubio, J. (2015) "An Updated Performance Comparison of Virtual Machines and Linux Containers", IEEE ISPASS, Philadelphia, PA.
- Khan, S. (2026) "Decomposing Docker Container Startup Performance: A Three-Tier Measurement Study on Heterogeneous Infrastructure", arXiv:2602.15214.

- Larabel, M. (2021) "Windows 11 WSL2 Performance Is Quite Competitive Against Ubuntu 20.04 LTS", Phoronix. Disponível em: <https://www.phoronix.com/review/windows11-wsl2-good>
- Microsoft (2023) "Comparing WSL Versions", Microsoft Learn. Disponível em: <https://learn.microsoft.com/en-us/windows/wsl/compare-versions>
- Potdar, A. M., Narayan, D. G., Kengond, S. and Mulla, M. M. (2020) "Performance Evaluation of Docker Container and Virtual Machine", Procedia Computer Science, vol. 171, pp. 1419–1428.
- Ruiz, C., Jeanvoine, E. and Nussbaum, L. (2015) "Performance Evaluation of Containers for HPC", IEEE/ACM CCGrid, Shenzhen, China.
- Sergeev, A., Rezedinova, E. and Khakhina, A. (2022) "Docker Container Performance Comparison on Windows and Linux Operating Systems", CIEES, Veliko Tarnovo.
- Stack Overflow (2024) "Stack Overflow Developer Survey 2024". Disponível em: <https://survey.stackoverflow.co/2024/>
- Szymanski, P. et al. (2024) "Docker Performance Evaluation across Operating Systems", Applied Sciences, vol. 14, no. 15, art. 6672, MDPI.